Predicting Heart Attack from Health Indicators: Building and Comparing Four Machine Learning Models

Student ID: [INSERT YOUR ID HERE]

Introduction

Heart attacks are a life-threatening emergency, and identifying high-risk people early can save lives. This report builds and compares four machine-learning models that predict whether a person has had heart disease or a heart attack from 15 self-reported health indicators, such as BMI, age and smoking status. It follows the stages of the brief: data preparation, building four models including an artificial neural network (ANN), evaluation, comparison, and improvement of the best model. All code is in the submitted notebook.

1.1 Data preparation

The dataset contains 129,998 respondents, 15 numeric features and a binary target, HeartDiseaseorAttack. Checking for missing values showed that only BMI was affected, with 10 missing entries (0.008%). Because this is negligible, those rows were deleted rather than imputed, leaving 129,988 records. All columns were already numeric, so no encoding was needed. Binary indicators (HighBP, Smoker, Sex), ordinal scales (Age in 13 bands, Education, Income) and counts (MentHlth, PhysHlth) were kept as supplied. I also found 22,961 duplicate rows and a maximum BMI of 98. Because the features are coarse categories, identical rows plausibly come from different people, and extreme BMI values are possible, so neither was removed.

Exploration (Figure 1) revealed the most important property of the data: it is highly imbalanced, with only 9.35% positive cases. A model that always predicts “no heart disease” would be 90.6% accurate while detecting nobody, so accuracy cannot be trusted alone. Age (r = 0.22), high blood pressure (0.21), stroke (0.21) and diabetes (0.18) correlate most positively with the target, whereas income (−0.14) and education (−0.10) correlate negatively, suggesting the features carry real signal.

The data were split 80/20 into 103,990 training and 25,998 test records (random_state=42). Stratification kept the 9.35% positive rate in both subsets, giving 2,431 positives in the test set. Features were standardised with StandardScaler, fitted on the training set only and then applied to the test set, which prevents information leaking from the test data. Scaling matters for logistic regression and the ANN and is harmless for tree-based models, so the same scaled matrices were used for all models to keep the comparison fair (Table 1).

Table 1. Data preparation summary

Step	Outcome
Missing values	10 missing BMI values (0.008%); rows removed → 129,988 records
Features / target	15 numeric features; binary target HeartDiseaseorAttack
Class balance	90.65% no heart disease, 9.35% heart disease or attack
Train/test split	80/20, stratified, random_state=42 → 103,990 train / 25,998 test
Scaling	StandardScaler fitted on training data only
Duplicates / outliers	22,961 duplicate rows and BMI up to 98 retained (plausible distinct respondents)

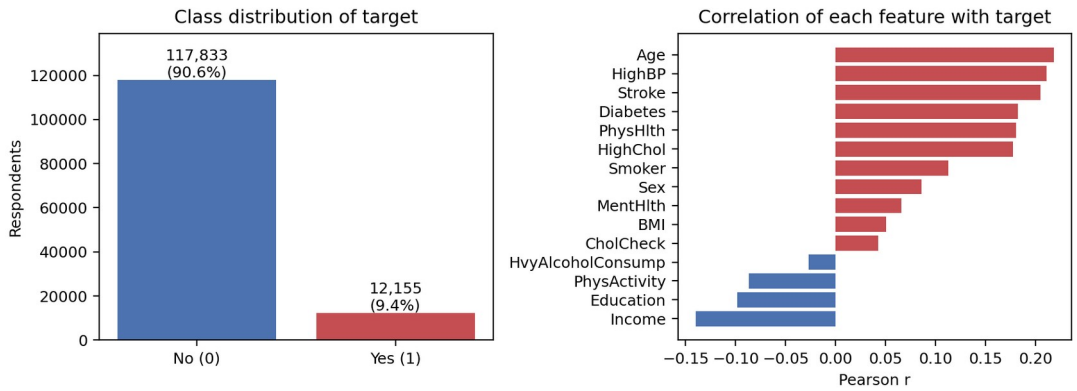


Figure 1. Class distribution of the target (left) and correlation of each feature with the target (right).

1.2 Choosing and tailoring four models

Four model families from the lectures were trained on identical data using scikit-learn (Pedregosa et al., 2011). Because of the imbalance, each was trained with and without class weighting, which penalises errors on the rare positive class more heavily (Table 2).

Logistic regression is a linear, interpretable baseline (max_iter=1000). A decision tree grown without limits memorises the training data, so I also trained a depth-limited tree (max_depth=5) and a depth-5 tree with balanced weights. The random forest (Breiman, 2001) averages many trees to reduce variance; the default forest used 200 trees, and the tailored version added max_depth=10 and min_samples_leaf=20 to limit overfitting, plus balanced weights.

The ANN is a Keras (Chollet, 2015) feed-forward network with 15 inputs, hidden layers of 64 and 32 ReLU neurons, each followed by 30% dropout for regularisation, and one sigmoid output giving the probability of heart disease. It used the Adam optimiser, binary cross-entropy loss and mini-batches of 256. Class weights (about 0.55 for negatives and 5.35 for positives) compensated for the imbalance. Twenty percent of the training data formed a validation set, and early stopping (patience 5, restoring the best weights) ended training at epoch 9 and returned the epoch-4 weights, where validation loss was lowest (0.502). Training and validation AUC stayed close, showing little overfitting.

Table 2. Model configurations

Model	Variants trained	Key settings
Logistic regression	Baseline; balanced	max_iter=1000, class_weight='balanced'
Decision tree	Default; depth 5; depth 5 balanced	max_depth=5, class_weight='balanced'
Random forest	Default; balanced	200 trees; max_depth=10, min_samples_leaf=20
ANN (Keras)	Balanced	64-32-1, ReLU, dropout 0.3, Adam, early stopping

1.3 Evaluation

Models were evaluated on the unseen test set. Accuracy is the share of correct predictions. Precision is the proportion of predicted heart-disease cases that are real, and recall (sensitivity) is the proportion of real cases found. F1 is their harmonic mean. The confusion matrix gives the counts of false negatives (missed patients) and false positives (false alarms). ROC AUC measures how well a model ranks people by risk regardless of threshold (Fawcett, 2006). For health screening a missed patient is far costlier than a false alarm, so recall, F1 and AUC were prioritised over accuracy.

Table 3 summarises all eight configurations. Three findings stand out. First, models trained without class weighting reached high accuracy (0.898–0.908) but very low recall (0.03–0.11, apart from the unrestricted tree at 0.27). Baseline logistic regression, for example, found only 230 of 2,431 patients. Second, the unrestricted tree overfitted, with an AUC of only 0.597, barely above chance. Third, class

weighting lifted recall to 0.73–0.83 in every model family, at the cost of precision (about 0.22) and accuracy (0.70–0.74). Figures 2 and 3 show the ROC curves and confusion matrices of the balanced models. The curves of logistic regression and the random forest almost overlap, and both sit clearly above the single tree. The confusion matrices show the trade-off directly: the ANN misses fewest patients (426) but raises the most false alarms (7,325), while logistic regression misses 528 and raises 6,340.

Table 3. Test-set results for all models (class 1 = heart disease; selected model highlighted)

Model	Accuracy	Precision	Recall	F1	ROC AUC
LogReg (baseline)	0.906	0.496	0.095	0.159	0.831
LogReg (balanced)	**0.736**	**0.231**	**0.783**	**0.357**	**0.832**
DecTree (default)	0.855	0.247	0.269	0.258	0.597
DecTree (max_depth=5)	0.908	0.611	0.032	0.060	0.798
DecTree (depth 5, balanced)	0.727	0.216	0.732	0.334	0.799
RandForest (default)	0.898	0.354	0.107	0.165	0.800
RandForest (balanced)	0.729	0.228	0.793	0.354	0.833
ANN (balanced)	0.702	0.215	0.825	0.341	0.835

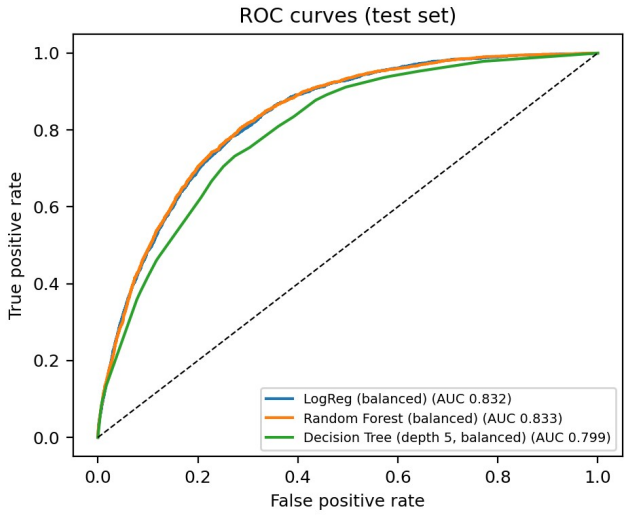


Figure 2. ROC curves of the balanced logistic regression, random forest and decision tree (the ANN's AUC of 0.835 is in Table 3).

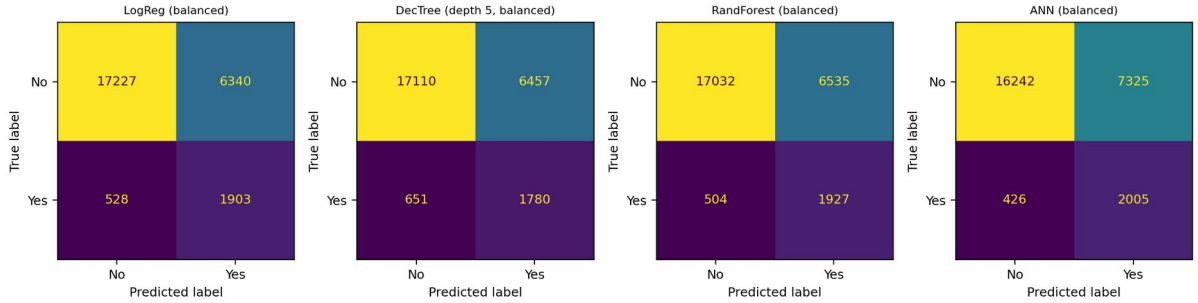


Figure 3. Confusion matrices of the four balanced models (rows = actual, columns = predicted).

1.4 Model comparison and best model

On accuracy alone, the depth-5 tree (0.908) and baseline logistic regression (0.906) rank first, but both merely match the 0.906 achieved by always predicting "no", so accuracy is not a valid basis for choosing the best model here. Among the four balanced models the differences are small. The ANN has the highest AUC (0.835) and recall (0.825), but also the most false alarms (7,325) and the lowest precision (0.215), giving F1 = 0.341. The random forest (AUC 0.833, recall 0.793, F1 0.354) and logistic regression

(AUC 0.832, recall 0.783, F1 0.357) are nearly identical. The tree is weakest (AUC 0.799, F1 0.334) because a single shallow tree cannot combine many weak risk factors.

To judge whether AUC differences are meaningful, 5-fold cross-validation was run for logistic regression. AUC ranged from 0.829 to 0.837 (mean 0.833), a spread larger than the 0.003 gap between logistic regression and the ANN. The ranking ability of logistic regression, the forest and the ANN is therefore indistinguishable.

Balanced logistic regression was selected as the best model. It has the highest F1 (0.357), detects 1,903 of 2,431 patients (78.3%), matches the complex models on AUC, trains in seconds and is interpretable. Its standardised coefficients show age as the dominant risk factor (+0.83), followed by sex (+0.40), high blood pressure (+0.35), high cholesterol (+0.33) and poor physical health (+0.26), while income is protective (−0.21). The random forest's feature importances agree, ranking age, blood pressure and cholesterol highest, which strengthens confidence in these findings. If maximising sensitivity were the only goal, the ANN would be preferable, but it flags 985 more healthy people to find just 102 additional patients.

1.5 Two ways to improve the best model

Both improvements were tested on the balanced logistic regression model, with every tuning decision made on training data only, never the test set.

Improvement 1: decision-threshold optimisation. The balanced model still flags 6,340 healthy people, because its default 0.5 cutoff is not optimised for F1. I generated out-of-fold probabilities for the balanced model using 5-fold cross-validation on the training set, and chose the cutoff that maximised F1 (0.677). The model was then refitted on the full training set and applied to the test set with that cutoff. Accuracy rose from 0.736 to 0.841, precision from 0.231 to 0.308 and F1 from 0.357 to 0.397, cutting false alarms from 6,340 to 3,062 while still detecting 1,362 patients (recall 0.560). AUC was unchanged, as expected, because only the cutoff moved, not the ranking. The cutoff can be lowered if sensitivity matters more than precision.

Improvement 2: feature engineering with interaction terms. Logistic regression is linear, so it cannot represent combinations such as age together with high blood pressure. Adding all degree-2 polynomial and interaction terms expanded 15 features to 135. Test AUC rose from 0.832 to 0.836 and recall from 0.783 to 0.805. To check this was not luck, 5-fold cross-validation showed mean AUC improving from 0.8332 to 0.8364, with the polynomial model higher in all five folds. At the default cutoff F1 stayed about the same (0.352), because the gain is in ranking quality.

Combining both (cutoff 0.704, chosen the same way) gave the best result: accuracy 0.848, precision 0.319, recall 0.551, F1 0.404 and AUC 0.836 (Table 4, Figure 4), a 13% relative F1 improvement over the balanced baseline. Two other ideas did not help: SMOTE oversampling (Chawla et al., 2002; F1 0.355, AUC 0.831) and regularisation tuning (best C=0.01, AUC unchanged at 0.832). This suggests the available features, not the algorithm, limit performance, so richer inputs such as clinical measurements would likely help most.

Table 4. Effect of the improvements on logistic regression (test set)

Configuration	Accuracy	Precision	Recall	F1	ROC AUC
Balanced baseline	0.736	0.231	0.783	0.357	0.832
+ Threshold tuning (0.677)	0.841	0.308	0.560	0.397	0.832
+ Polynomial features	0.724	0.226	0.805	0.352	0.836
+ Both improvements (0.704)	**0.848**	**0.319**	**0.551**	**0.404**	**0.836**
SMOTE oversampling (no gain)	0.736	0.230	0.779	0.355	0.831

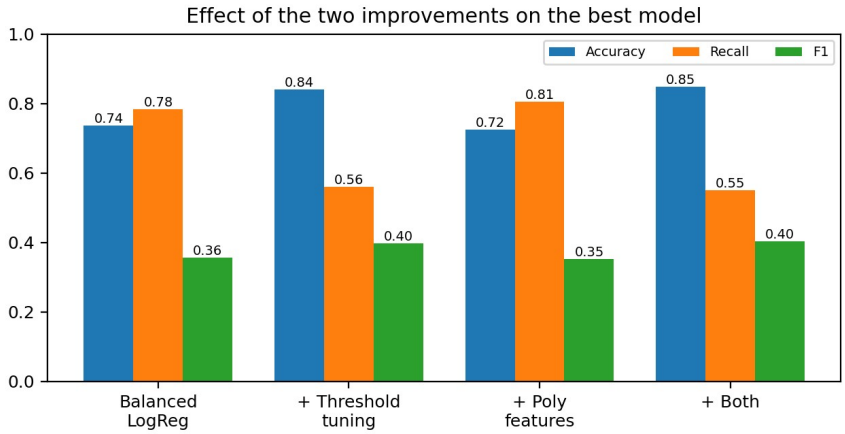


Figure 4. Accuracy, recall and F1 for the best model before and after each improvement.

Conclusion

Four models were built and compared on an imbalanced dataset of 129,988 respondents. Accuracy was misleading, so recall, F1 and AUC guided selection. Balanced logistic regression was the best overall model (recall 0.783, F1 0.357, AUC 0.832), and threshold tuning with interaction features lifted F1 to 0.404. Because the data are self-reported survey responses, the model supports screening rather than clinical diagnosis. Future work could add gradient boosting, choose the threshold using real clinical costs of missed cases versus false alarms, and validate the model on an independent population.

References

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357. https://doi.org/10.1613/jair.953

Chollet, F. (2015). Keras [Computer software]. https://keras.io

Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[Dataset author/organisation]. ([Year]). heart_attack.csv [Data set]. Moodle, [Module code]. [Complete from your Moodle page]